

1. Estructura de Carpetas Actualizada

La estructura de archivos sigue siendo la misma de la implementación anterior. Solo se ha modificado `src/core/states.py` para incluir los nuevos estados.

```
...
el-ultimo-bastion/
├── main.py          (sin cambios)
├── config.py        (sin cambios)
├── src/
│   └── core/
│       ├── __init__.py (sin cambios)
│       ├── game.py     (sin cambios)
│       └── states.py    ← MODIFICADO
...
```

No se han creado nuevas carpetas ni archivos adicionales. El resto de directorios (`entities`, `managers`, `map`, `ui`, `utils`, `assets`, ...) permanecen vacíos a la espera de futuras implementaciones.

2. Archivos Modificados y Código Completo

`src/core/states.py` (versión completa y actualizada)

```
```python
"""
Máquina de estados del juego "El Último Bastión".
Define la clase base State y las implementaciones concretas para:
- Menú Principal (MenuState)
- Juego en desarrollo (GameplayState)
- Pausa (PauseState)
- Game Over (GameOverState)
"""

import pygame
from config import COLOR_DARK_BG, COLOR_MENU_TEXT, COLOR_WHITE,
COLOR_BLUE, COLOR_RED, COLOR_BLACK

class State:
 """
 Clase base abstracta para todos los estados del juego.
 Define la interfaz común que deben implementar los estados concretos.
 """
 def __init__(self, game):
 """
 Args:

```

```

 game: referencia a la instancia principal de Game.
 """
 self.game = game

 def handle_events(self, event):
 """Procesa un evento de Pygame."""
 raise NotImplementedError

 def update(self, dt):
 """Actualiza la lógica del estado. dt en segundos."""
 raise NotImplementedError

 def render(self, screen):
 """Dibuja el estado en la superficie dada."""
 raise NotImplementedError

class MenuState(State):
 """
 Pantalla de menú principal.
 Muestra las opciones 'Jugar' y 'Salir'.
 """
 def __init__(self, game):
 super().__init__(game)
 # Fuentes
 self.title_font = pygame.font.Font(None, 72)
 self.option_font = pygame.font.Font(None, 48)

 # Textos del menú
 self.title_text = self.title_font.render("EL ÚLTIMO BASTIÓN", True,
COLOR_MENU_TEXT)
 self.play_text = self.option_font.render("Presiona ENTER para Jugar", True,
COLOR_WHITE)
 self.quit_text = self.option_font.render("Presiona ESC para Salir", True,
COLOR_WHITE)

 # Posiciones
 screen_center_x = self.game.screen.get_width() // 2
 screen_center_y = self.game.screen.get_height() // 2

 self.title_rect = self.title_text.get_rect(center=(screen_center_x, screen_center_y - 100))
 self.play_rect = self.play_text.get_rect(center=(screen_center_x, screen_center_y +
20))
 self.quit_rect = self.quit_text.get_rect(center=(screen_center_x, screen_center_y + 80))

 def handle_events(self, event):
 if event.type == pygame.KEYDOWN:
 if event.key == pygame.K_RETURN:

```

```

 # Iniciar el juego (crea un nuevo GameplayState)
 self.game.change_state(GameplayState(self.game))
 elif event.key == pygame.K_ESCAPE:
 self.game.running = False # Salir de la aplicación

def update(self, dt):
 pass

def render(self, screen):
 screen.fill(COLOR_DARK_BG)
 screen.blit(self.title_text, self.title_rect)
 screen.blit(self.play_text, self.play_rect)
 screen.blit(self.quit_text, self.quit_rect)

class GameplayState(State):
 """
 Estado principal de juego (placeholder).
 En esta versión solo muestra un fondo y un texto informativo.
 Al presionar ESC se activa la pausa.
 """
 def __init__(self, game):
 super().__init__(game)
 self.font = pygame.font.Font(None, 36)
 self.info_text = self.font.render("Juego en desarrollo... (ESC para pausa)", True,
COLOR_WHITE)
 self.info_rect = self.info_text.get_rect(center=(self.game.screen.get_width()/2,
self.game.screen.get_height()/2))

 def handle_events(self, event):
 if event.type == pygame.KEYDOWN:
 if event.key == pygame.K_ESCAPE:
 # Pausar: pasar a PauseState guardando referencia a esta misma instancia
 self.game.change_state(PauseState(self.game, self))

 def update(self, dt):
 # En el futuro actualizará enemigos, defensas, oleadas...
 pass

 def render(self, screen):
 # Fondo azul (placeholder del escenario de juego)
 screen.fill(COLOR_BLUE)
 screen.blit(self.info_text, self.info_rect)

class PauseState(State):
 """
 Pantalla de pausa.

```

Se superpone al estado de juego (GameplayState) y ofrece opciones para continuar o volver al menú principal.

```
"""
def __init__(self, game, gameplay_state):
 """
 Args:
 game: instancia principal de Game.
 gameplay_state: instancia de GameplayState que está en pausa.
 """
 super().__init__(game)
 self.gameplay_state = gameplay_state # Guardamos para reanudar

 # Fuentes
 self.title_font = pygame.font.Font(None, 64)
 self.option_font = pygame.font.Font(None, 42)

 self.title_text = self.title_font.render("PAUSA", True, COLOR_WHITE)
 self.continue_text = self.option_font.render("Presiona C para Continuar", True,
COLOR_WHITE)
 self.menu_text = self.option_font.render("Presiona M para Volver al Menú", True,
COLOR_WHITE)

 # Posiciones centradas
 screen_center_x = game.screen.get_width() // 2
 screen_center_y = game.screen.get_height() // 2

 self.title_rect = self.title_text.get_rect(center=(screen_center_x, screen_center_y - 80))
 self.continue_rect = self.continue_text.get_rect(center=(screen_center_x,
screen_center_y))
 self.menu_rect = self.menu_text.get_rect(center=(screen_center_x, screen_center_y +
60))

def handle_events(self, event):
 if event.type == pygame.KEYDOWN:
 if event.key == pygame.K_c:
 # Reanudar: volver al estado de juego guardado
 self.game.change_state(self.gameplay_state)
 elif event.key == pygame.K_m:
 # Volver al menú principal (descarta la partida actual)
 self.game.change_state(MenuState(self.game))

def update(self, dt):
 # No se actualiza nada durante la pausa
 pass

def render(self, screen):
 # Primero dibujamos el estado de juego congelado (fondo)
 self.gameplay_state.render(screen)
```

```

Luego una capa semitransparente oscura
overlay = pygame.Surface(screen.get_size(), pygame.SRCALPHA)
overlay.fill((0, 0, 0, 180)) # Negro con alpha 180
screen.blit(overlay, (0, 0))

```

```

Textos de pausa encima
screen.blit(self.title_text, self.title_rect)
screen.blit(self.continue_text, self.continue_rect)
screen.blit(self.menu_text, self.menu_rect)

```

```

class GameOverState(State):

```

```

 """

```

```

 Pantalla de Game Over.
 Se muestra cuando el jugador pierde la partida.
 Permite reiniciar o salir al menú principal.
 """

```

```

 def __init__(self, game):

```

```

 super().__init__(game)

```

```

 # Fuentes

```

```

 self.title_font = pygame.font.Font(None, 72)

```

```

 self.option_font = pygame.font.Font(None, 42)

```

```

 self.title_text = self.title_font.render("GAME OVER", True, COLOR_RED)

```

```

 self.restart_text = self.option_font.render("Presiona R para Reiniciar", True,
COLOR_WHITE)

```

```

 self.menu_text = self.option_font.render("Presiona S para Salir al Menú", True,
COLOR_WHITE)

```

```

 # Posiciones

```

```

 screen_center_x = game.screen.get_width() // 2

```

```

 screen_center_y = game.screen.get_height() // 2

```

```

 self.title_rect = self.title_text.get_rect(center=(screen_center_x, screen_center_y - 80))

```

```

 self.restart_rect = self.restart_text.get_rect(center=(screen_center_x, screen_center_y))

```

```

 self.menu_rect = self.menu_text.get_rect(center=(screen_center_x, screen_center_y +
60))

```

```

 def handle_events(self, event):

```

```

 if event.type == pygame.KEYDOWN:

```

```

 if event.key == pygame.K_r:

```

```

 # Reiniciar: crea una nueva partida

```

```

 self.game.change_state(GameplayState(self.game))

```

```

 elif event.key == pygame.K_s:

```

```

 # Salir al menú

```

```

 self.game.change_state(MenuState(self.game))

```

```

def update(self, dt):
 pass

def render(self, screen):
 # Fondo oscuro
 screen.fill(COLOR_BLACK)
 screen.blit(self.title_text, self.title_rect)
 screen.blit(self.restart_text, self.restart_rect)
 screen.blit(self.menu_text, self.menu_rect)
...

```

### Archivos sin cambios

```

- `main.py`
- `config.py`
- `src/core/__init__.py`
- `src/core/game.py`

```

Estos permanecen idénticos a la entrega anterior (no requieren modificación para la gestión de estados).

---

### ## 3. Explicación Técnica del Funcionamiento

#### ### Gestión de estados

La clase `Game` (en `game.py`) mantiene una referencia al estado activo mediante `self.state`. El método `change\_state(new\_state)` simplemente reasigna esta variable. Gracias al polimorfismo, el bucle principal no necesita conocer qué estado concreto está activo: llama a `handle\_events()`, `update(dt)` y `render(screen)` de forma uniforme.

Todos los estados heredan de `State`, que declara los tres métodos abstractos. Cada estado implementa su propia lógica de eventos, actualización y renderizado.

#### ### Cambio entre escenas y navegación por teclado

Cada escena se corresponde con una subclase de `State` y define transiciones a otras escenas mediante la creación de nuevos estados y la llamada a `game.change\_state(...)`. Las transiciones están activadas por teclas:

Estado actual	Tecla	Nuevo estado	Observaciones
-----	-----	-----	-----
`MenuState`	`ENTER`	`GameplayState`	Inicia una partida nueva
`MenuState`	`ESC`	(salir)	`game.running = False`, termina la aplicación
`GameplayState`	`ESC`	`PauseState`	Pasa el propio `GameplayState` a `PauseState`
`PauseState`	`C`	`GameplayState`	Reanuda la partida guardada
`PauseState`	`M`	`MenuState`	Vuelve al menú (descarta la partida)

`GameOverState`   `R`	`GameplayState`	Reinicia la partida (nueva instancia)
`GameOverState`   `S`	`MenuState`	Sale al menú principal

Además, el botón de cierre de la ventana (evento `QUIT`) siempre detiene la aplicación, independientemente del estado, porque se procesa en `Game.handle\_events()` antes de delegar al estado.

#### ### Integración con el Game Loop existente

El bucle principal en `Game.run()` no se ha modificado: sigue calculando el delta time, llamando a `handle\_events()`, `update(dt)` y `render()`. Como cada estado implementa estos métodos, el bucle funciona exactamente igual para menú, juego, pausa y game over. La única precaución es que durante la pausa y el game over, la lógica del juego no avanza porque el `GameplayState` no está activo (no recibe `update`).

#### ### Renderizado durante la pausa

Para dar realimentación visual de que el juego está congelado, `PauseState` recibe el `GameplayState` original como argumento. Su método `render()`:

1. Llama a `self.gameplay\_state.render(screen)` para pintar el escenario de juego exactamente como estaba.
2. Dibuja una superficie semitransparente oscura sobre toda la pantalla.
3. Superpone los textos del menú de pausa.

Esto hace que el fondo del juego se vea oscurecido y se lea claramente la información de pausa, sin modificar la lógica de renderizado del `GameplayState`.

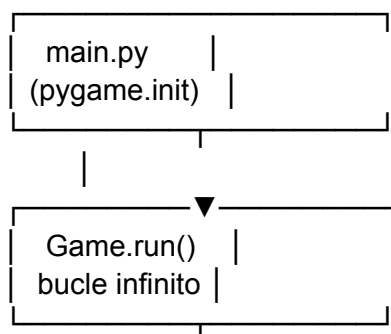
#### ### Conservación del estado de la partida

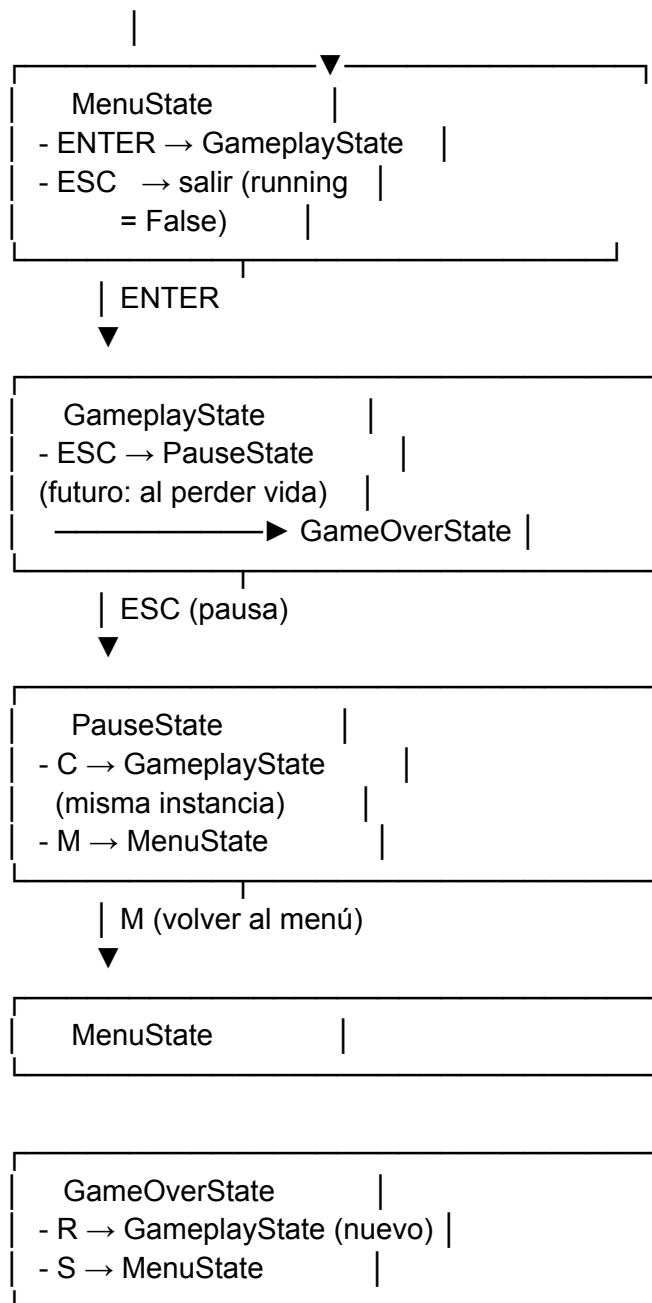
Cuando se pausa, se guarda la misma instancia de `GameplayState` en `PauseState.gameplay\_state`. Al reanudar, se restaura esa misma instancia, por lo que cualquier lógica que en el futuro almacene (oleadas, recursos, posición de enemigos) se mantendrá intacta. Al volver al menú o al reiniciar, se crea un nuevo `GameplayState`, lo que reinicia completamente la partida.

---

### ## 4. Diagrama Textual del Flujo de Estados

...





...

*\*Nota:\** La transición desde `GameplayState` hacia `GameOverState` aún no está implementada (requiere lógica de derrota), pero queda reflejada para completitud del diseño.

---

## ## 5. Instrucciones de Ejecución (sin cambios)

1. Asegúrate de tener Python 3 y Pygame instalados (`pip install pygame`).
2. Coloca los archivos según la estructura indicada.
3. Desde la carpeta `el-ultimo-bastion/`, ejecuta `python main.py`.
4. Usa el teclado para navegar entre las escenas tal como se describe en la tabla de arriba.



Este sistema de estados cumple con los principios de modularidad y escalabilidad de la arquitectura original, y está listo para integrar los futuros componentes de juego (enemigos, torres, oleadas) dentro de `GameplayState`.